Operating Systems

No. 8

Sarun Intakosum

Real Memory Management

Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
  - **Compile time**: If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
  - **Load time**: Must generate **relocatable code** if memory location is not known at compile time
  - **Execution time**: Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., base and limit registers)

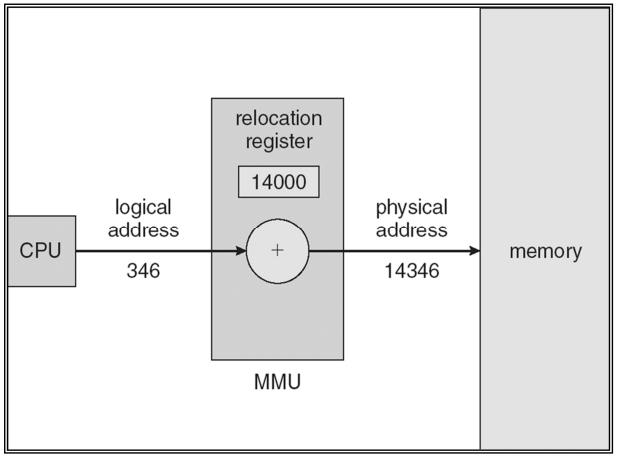
Logical vs. Physical Address Space

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

Memory-Management Unit (MMU)

- Hardware device that maps virtual to physical address
- In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program deals with *logical* addresses; it never sees the *real* physical addresses

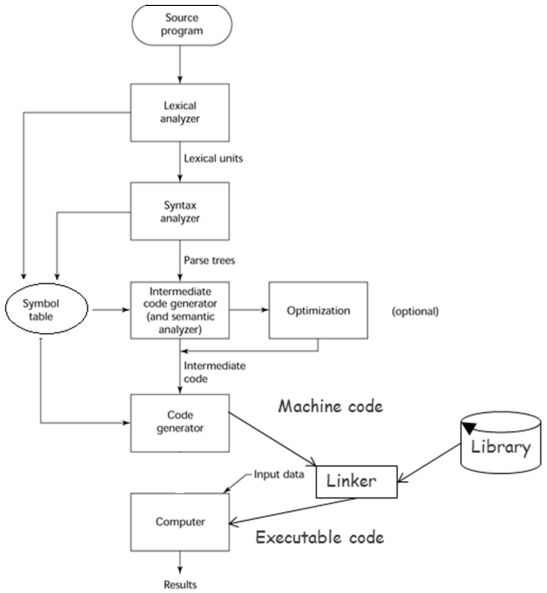
Dynamic relocation using a relocation register



Dynamic Linking

- Linking postponed until execution time
- Small piece of code, *stub*, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system needed to check if routine is in processes' memory address
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**

Linking

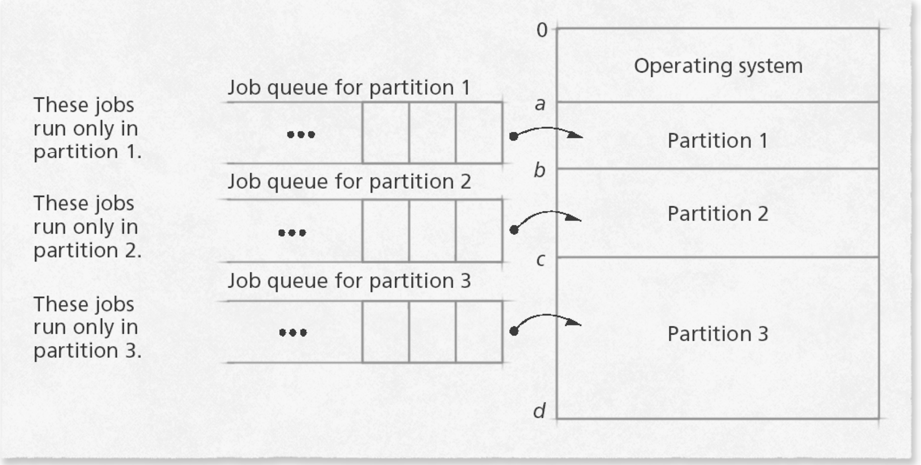


Fixed Partitions

- **Fixed Partitions:** Main memory is partitioned; one partition/job
  - Allows multiprogramming
  - Partition sizes remain static unless and until computer system id shut down, reconfigured, and restarted
  - Requires protection of the job's memory space
  - Requires matching job size with partition size

Fixed-Partition Multiprogramming

Fixed-partition multiprogramming with absolute translation and loading.

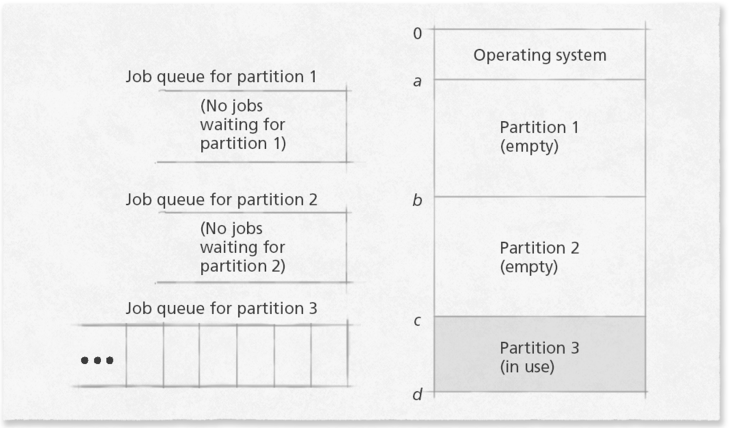


Fixed-Partition Multiprogramming

- Drawbacks to fixed partitions
  - Early implementations used absolute addresses
    - If the requested partition was full, code could not load
    - Later resolved by relocating compilers

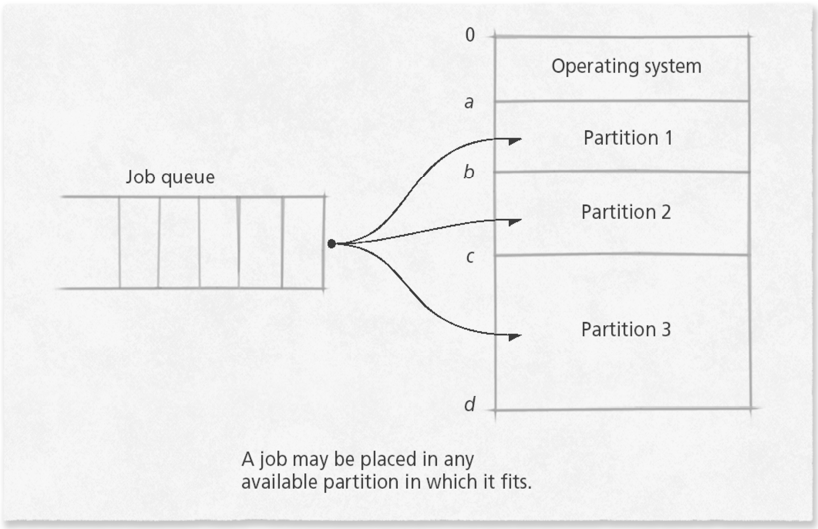
Fixed-Partition Multiprogramming

Memory waste under fixed-partition multiprogramming with absolute translation and loading.



Fixed-Partition Multiprogramming

Fixed-partition multiprogramming with relocatable translation and loading.



Fixed Partitions (continued)

To allocate memory spaces to jobs, the operating system's Memory Manager must keep a table as shown below:

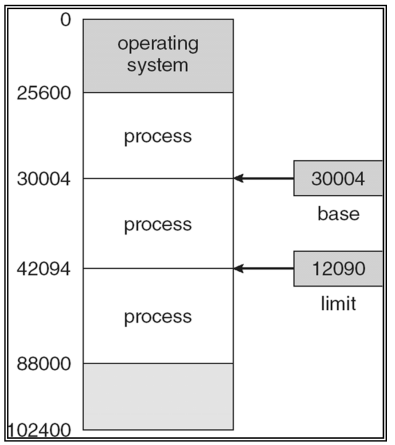
Partition Size	Memory Address	Access	Partition Status
100K	200K	Job 1	Busy
25K	300K	Job 4	Busy
25K	325K		Free
50K	350K	Job 2	Busy

Fixed-Partition Multiprogramming

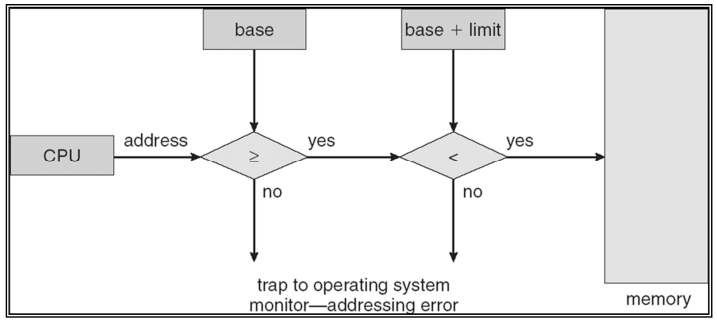
- Protection
  - Can be implemented by boundary registers, called base and limit (also called low and high)

Base and Limit Registers

- A pair of **base** and **limit** registers define the logical address space

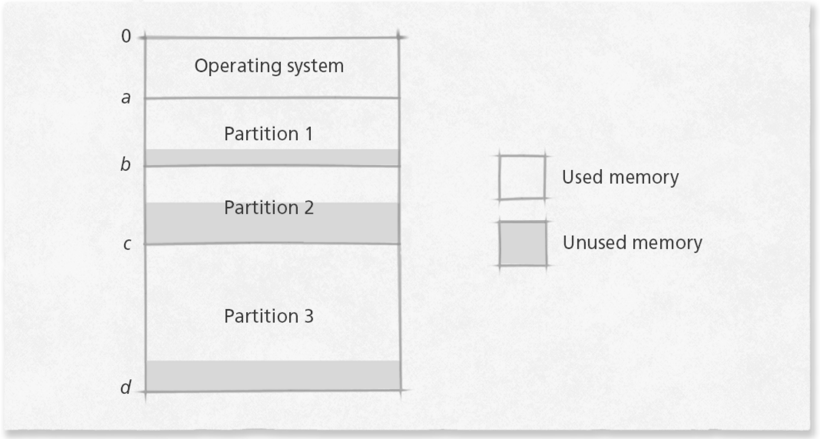


HW address protection with base and limit registers

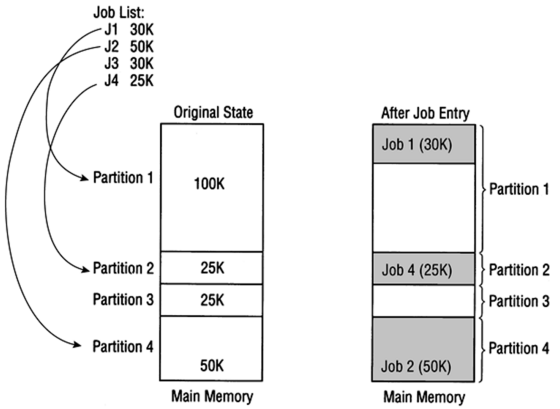


Fixed-Partition Multiprogramming

Internal fragmentation in a fixed-partition multiprogramming system.



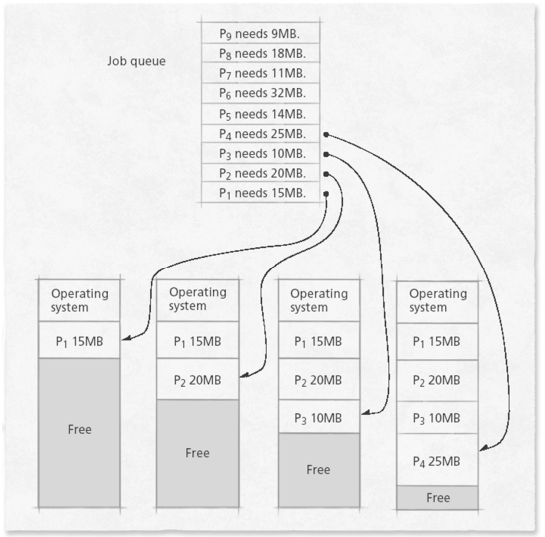
Fixed Partitions (continued)



NOTE: Job 3 must wait even though 70K of free space is available in Partition 1 where Job 1 occupies only 30K of the 100K available

Variable-Partition Multiprogramming

Initial partition assignments in variable-partition programming.

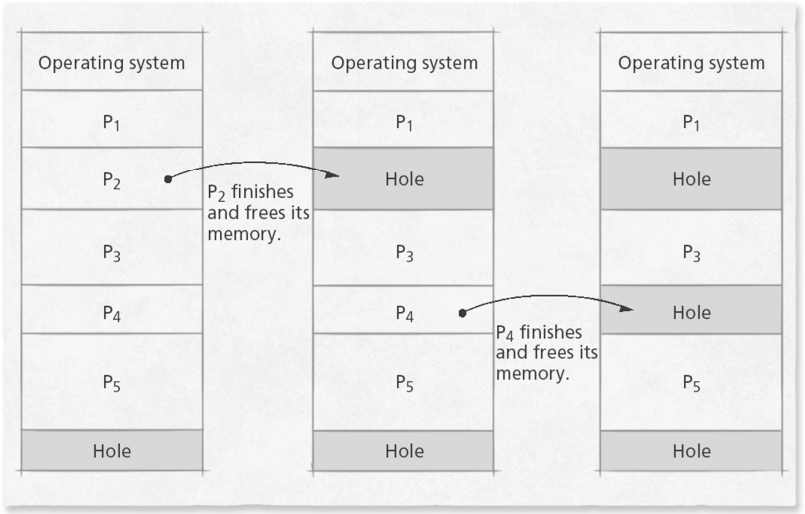


Variable-Partition Characteristics

- Jobs placed where they fit
  - No space wasted initially
  - Internal fragmentation impossible
    - Partitions are exactly the size they need to be
  - External fragmentation can occur when processes removed
    - Leave holes too small for new processes
    - Eventually no holes large enough for new processes

Variable-Partition Characteristics

Memory "holes" in variable-partition multiprogramming.

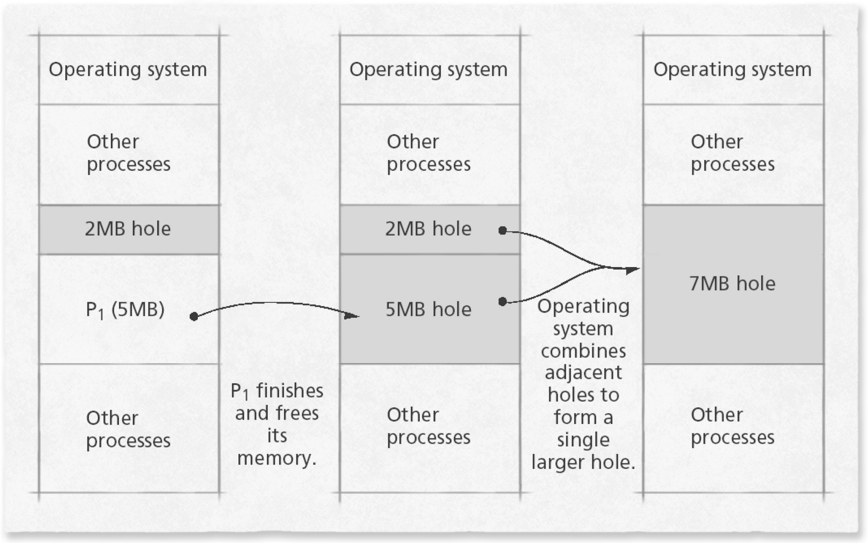


Variable-Partition Characteristics

- Several ways to combat external fragmentation
  - Coalescing
    - Combine adjacent free blocks into one large block
    - Often not enough to reclaim significant amount of memory
  - Compaction
    - Sometimes called garbage collection (not to be confused with GC in object-oriented languages)
    - Rearranges memory into a single contiguous block free space and a single contiguous block of occupied space
    - Makes all free space available
    - Significant overhead
    - Compaction is possible *only* if relocation is dynamic, and is done at execution time

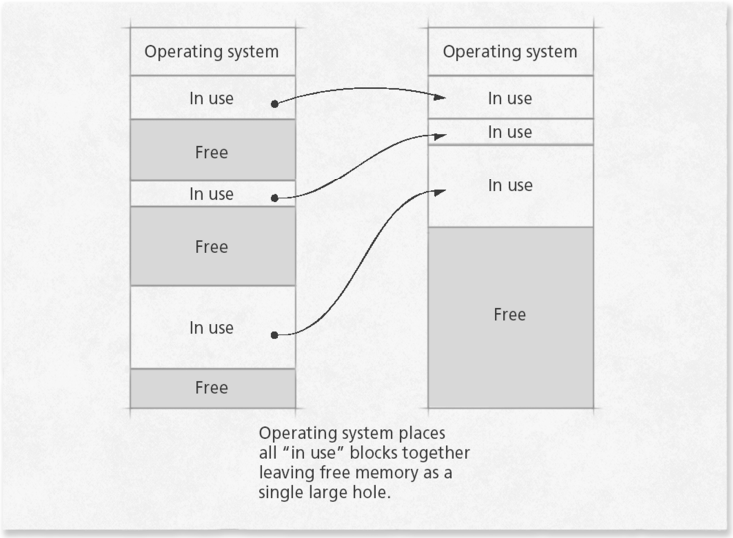
Variable-Partition Characteristics

Coalescing memory "holes" in variable-partition multiprogramming.



Variable-Partition Characteristics

Memory compaction in variable-partition multiprogramming.

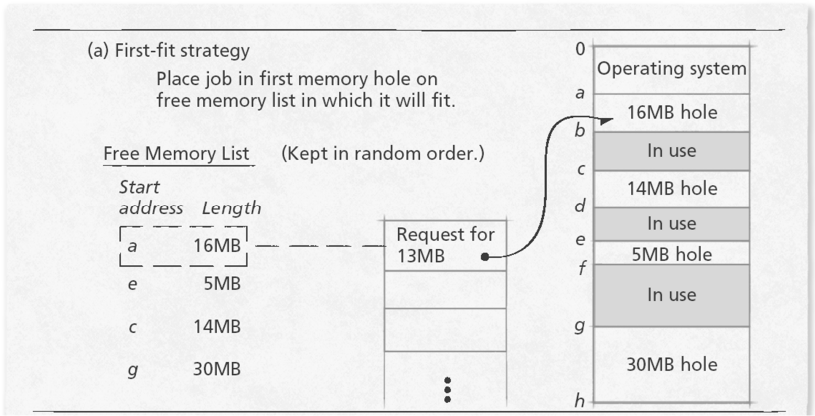


Memory Placement Strategies

- Where to put incoming processes
  - First-fit strategy
    - Process placed in first hole of sufficient size found
    - Simple, low execution-time overhead
  - Best-fit strategy
    - Process placed in hole that leaves least unused space around it
    - More execution-time overhead
  - Worst-fit strategy
    - Process placed in hole that leaves most unused space around it
    - Leaves another large hole, making it more likely that another process can fit in the hole

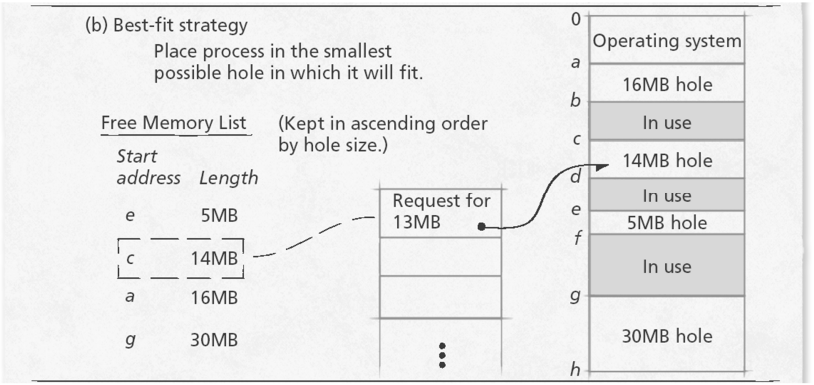
Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 1 of 3).



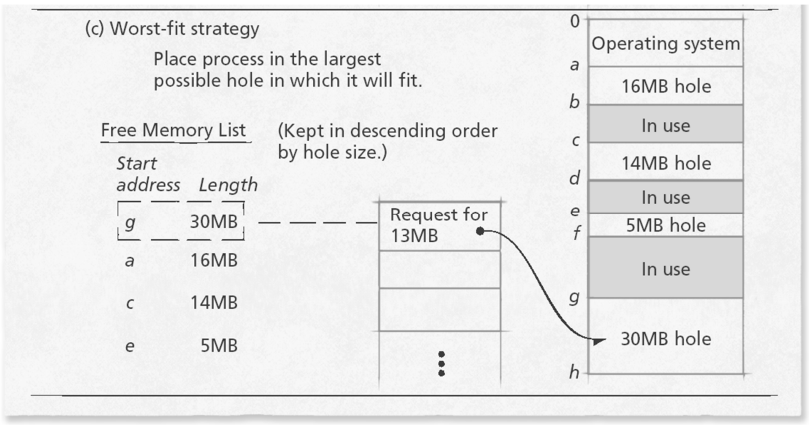
Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 2 of 3).



Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 3 of 3).



Best-Fit Versus First-Fit Allocation (continued)

- **First-fit memory allocation:**
  - **Advantage:** Faster in making allocation
  - **Disadvantage:** Leads to memory waste
- **Best-fit memory allocation**
  - **Advantage:** Makes the best use of memory space
  - **Disadvantage:** Slower in making allocation

Variable Partitions

- **Disadvantages:**
  - Fully utilizes memory only when the first jobs are loaded
  - Subsequent allocation leads to memory waste or external fragmentation

Noncontiguous Scheme

Paging

- Paging is noncontiguous memory allocation scheme
- Divide physical memory into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8,192 bytes)
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size ***n*** pages, need to find *n* free frames and load program
- Set up a page table to translate logical to physical addresses
- Internal fragmentation

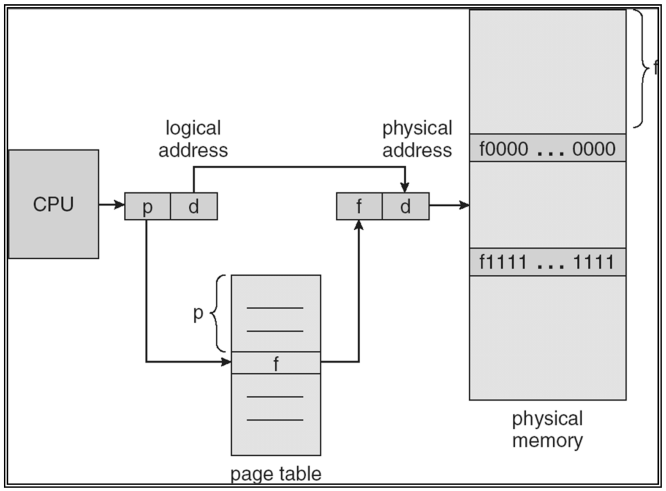
Address Translation Scheme

- Address (m bits) generated by CPU is divided into:
  - **Page number (*p*)** – used as an index into a *page table* which contains base address of each page in physical memory
  - **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit

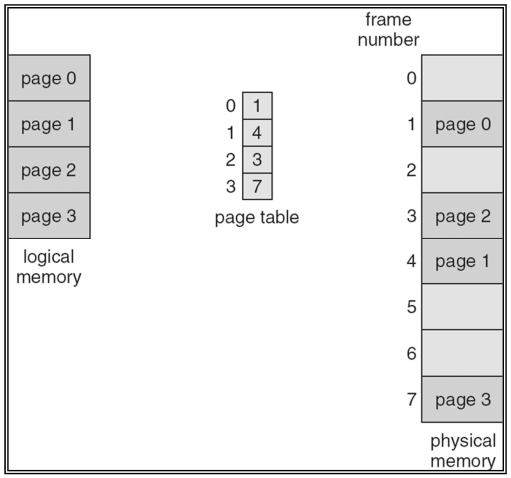
page number	page offset
p	*d*
m - n	*n*

- For given logical address space 2^m *and page size* 2^n

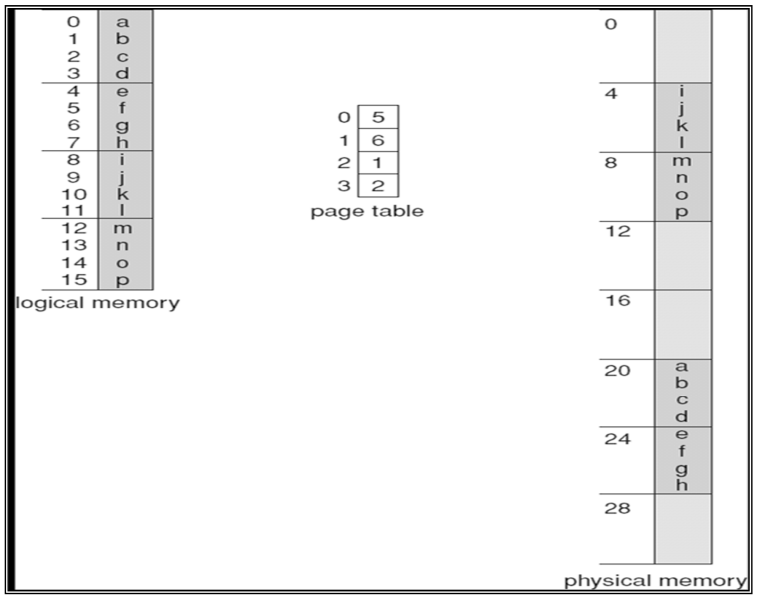
Paging Hardware



Paging Model of Logical and Physical Memory

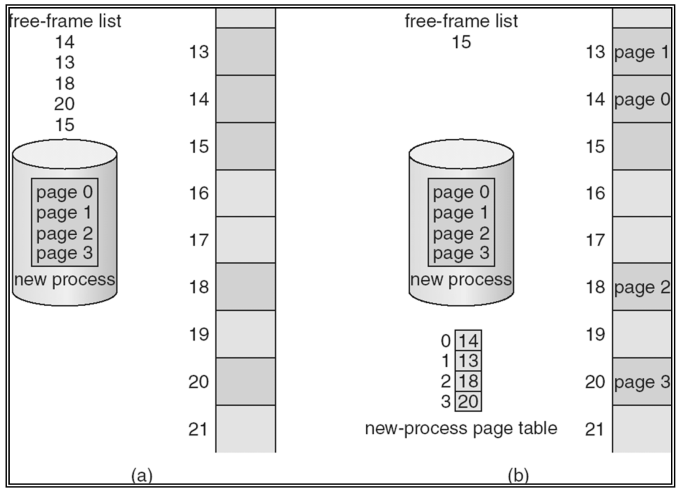


Paging Example



32-byte memory and 4-byte pages

Free Frames



Before allocation

After allocation

Implementation of Page Table

- Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses. One for the page table and one for the data/instruction.
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

Associative Memory

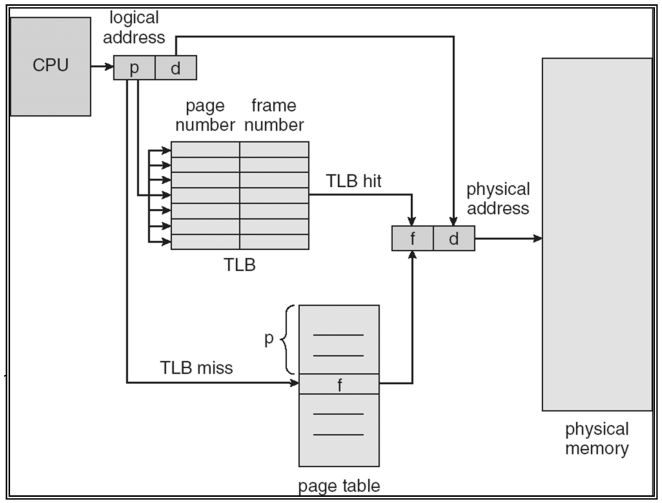
- Associative memory – parallel search

Page #	Frame #

Address translation (p, d)

- If p is in associative register, get frame # out
- Otherwise get frame # from page table in memory

Paging Hardware With TLB



Effective Access Time

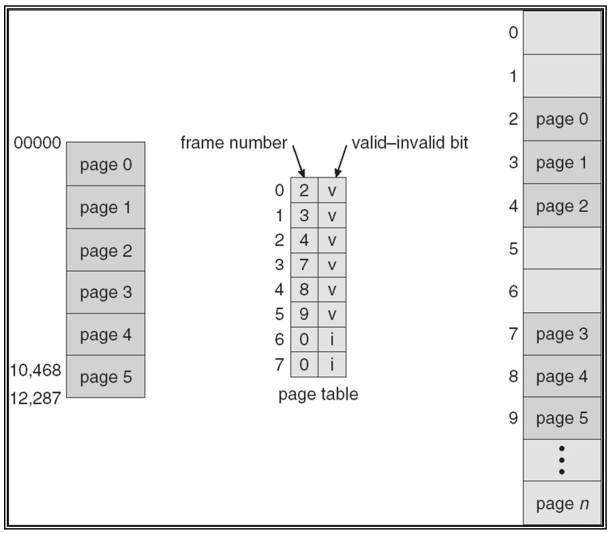
- Associative Lookup = ε time unit
- Assume memory cycle time is 1 microsecond
- Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers
- Hit ratio = α
- **Effective Access Time** (EAT)

$$EAT = (1 + \varepsilon)\,\alpha + (2 + \varepsilon)(1 - \alpha)$$
$$= 2 + \varepsilon - \alpha$$

Memory Protection

- Memory protection implemented by associating protection bit with each frame
- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space

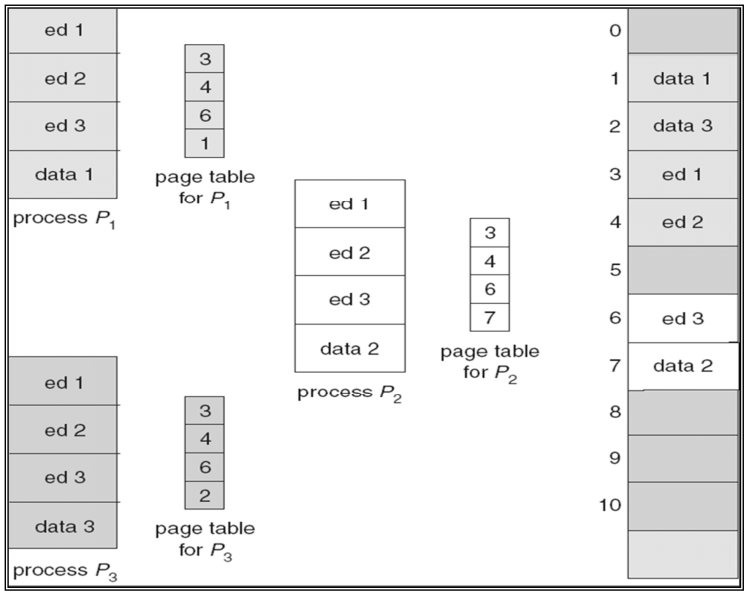
Valid (v) or Invalid (i) Bit In A Page Table



Shared Pages

- **Shared code**
  - One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems).
  - Shared code must appear in same location in the logical address space of all processes
- **Private code and data**
  - Each process keeps a separate copy of the code and data
  - The pages for the private code and data can appear anywhere in the logical address space

Shared Pages Example



Structure of the Page Table

- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table

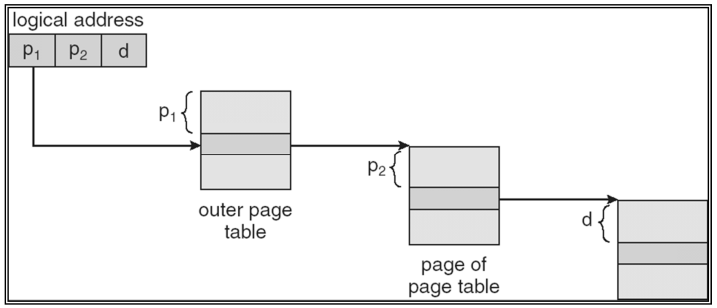
Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
  - a page number consisting of 22 bits
  - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
  - a 12-bit page number
  - a 10-bit page offset
- Thus, a logical address is as follows:

page number		page offset
p_i	p_2	d
12	10	10

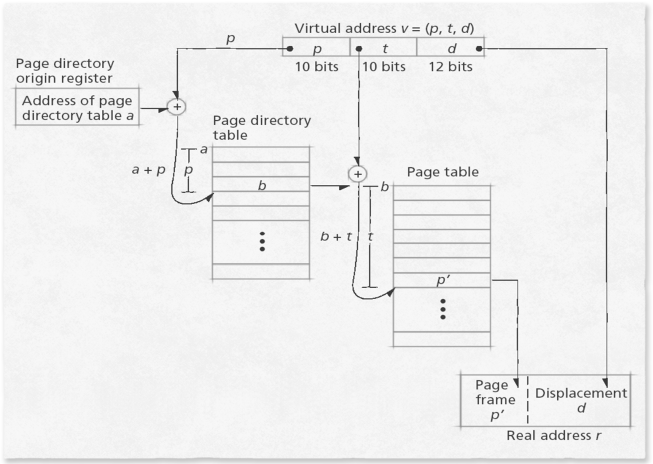
where p_i is an index into the outer page table, and p_2 is the displacement within the page of the outer page table

Address-Translation Scheme



Multilevel Page Tables

Multilevel page address translation.



Three-level Paging Scheme

outer page	inner page	offset
p_1	p_2	d
42	10	12

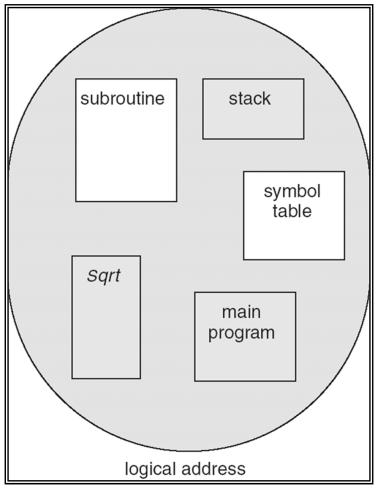
2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

Segmentation

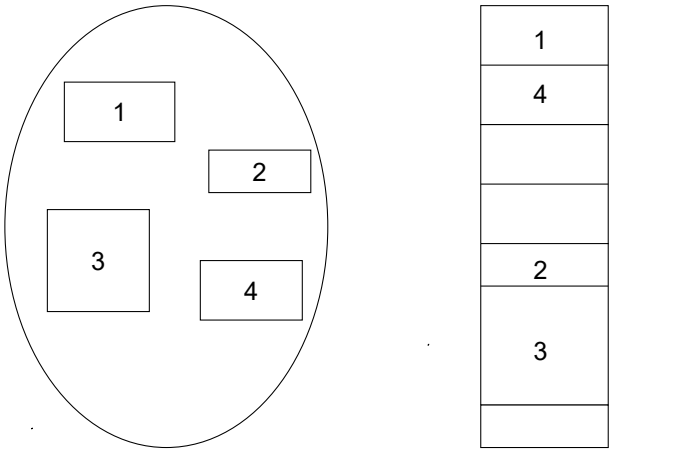
- Memory-management scheme that supports user view of memory
- A program is a collection of segments. A segment is a logical unit such as:

  main program,
  procedure,
  function,
  method,
  object,
  local variables, global variables,
  common block,
  stack,
  symbol table, arrays

User's View of a Program



Logical View of Segmentation



user space

physical memory space

Segmentation Architecture

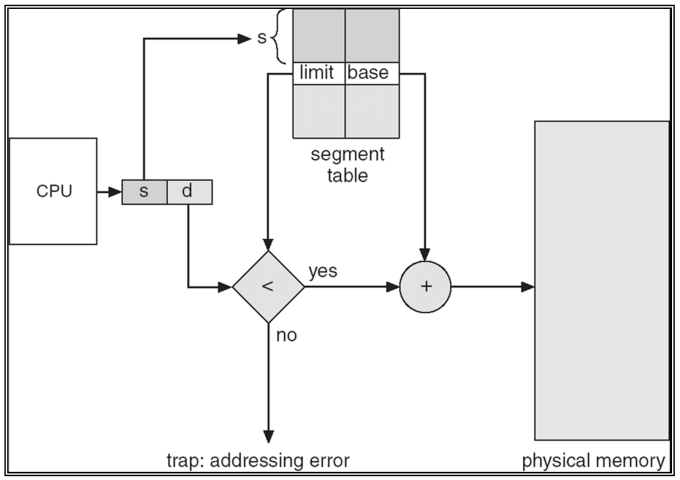
- Logical address consists of a two tuple:

  <segment-number, offset>,

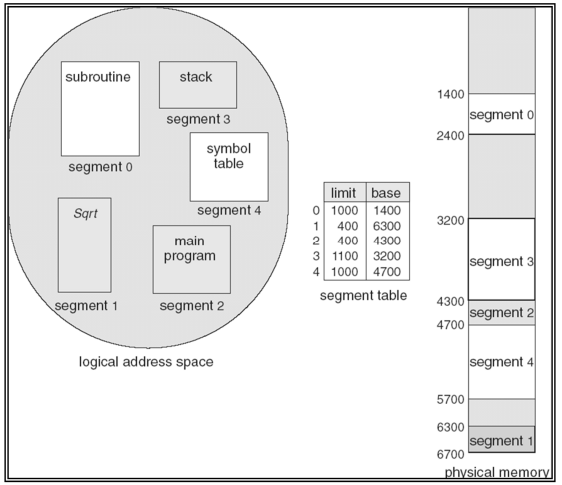
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
  - **base** – contains the starting physical address where the segments reside in memory
  - **limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;

  segment number ***s*** is legal if ***s*** < **STLR**

Segmentation Hardware



Example of Segmentation



Segmentation/Paging Systems

- Process references virtual memory address

  $v = (s, p, d)$

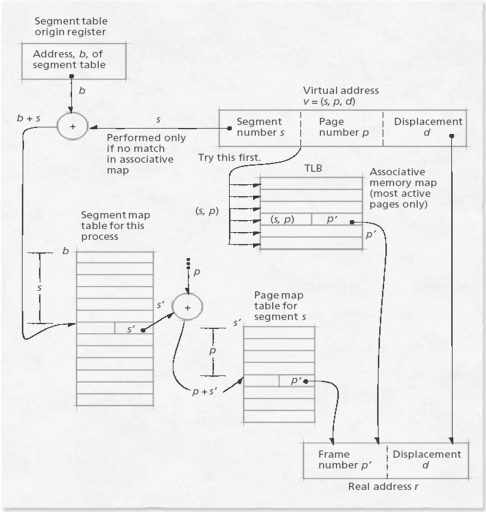
  - DAT adds the process's segment map table base address, b, to referenced segment number, s
  - $b + s$ forms the main memory address of the segment map table entry for segment s
  - Segment map table entry stores the base address of the page table, s'
  - Referenced page number, p, is added to s' to locate the PTE for page p, which stores page frame number p'
  - System concatenates p' with displacement, d, to form real address, r

Dynamic Address Translation in a Segmentation/Paging System

Virtual address format in a segmentation/paging system.

Dynamic Address Translation in a Segmentation/Paging System

Virtual address translation with combined associative/direct mapping in a segmentation/paging system.

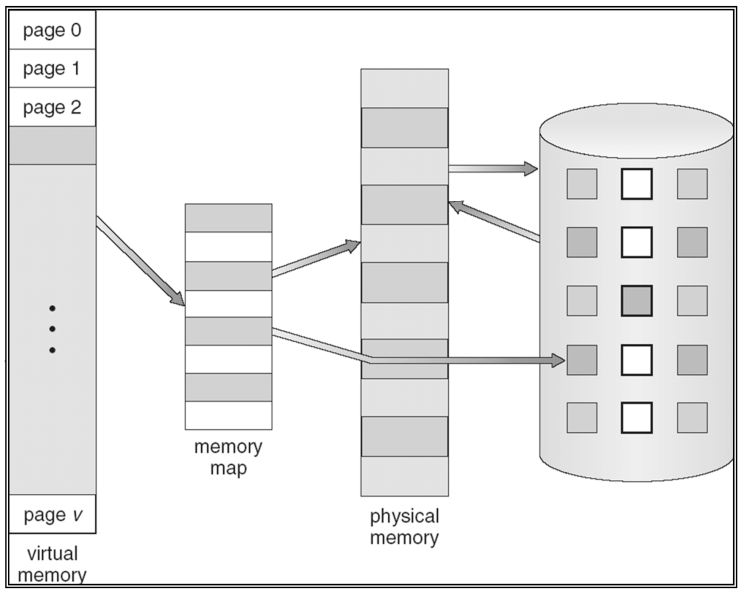


Virtual-Memory Management

Background

- **Virtual memory** – separation of user logical memory from physical memory.
  - Only part of the program needs to be in memory for execution
  - Logical address space can therefore be much larger than physical address space
  - Allows address spaces to be shared by several processes
  - Allows for more efficient process creation
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation

Virtual Memory That is Larger Than Physical Memory

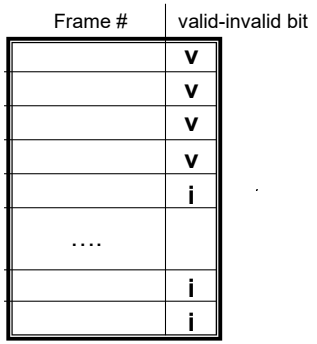


Demand Paging

- Bring a page into memory only when it is needed
  - Less I/O needed
  - Less memory needed
  - Faster response
  - More users
- Page is needed ⇒ reference to it
  - invalid reference ⇒ abort
  - not-in-memory ⇒ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
  - Swapper that deals with pages is a **pager**

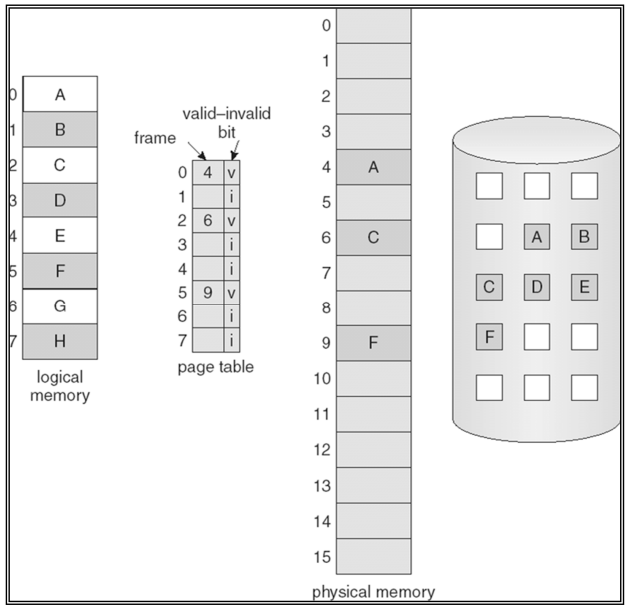
Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** ⇒ in-memory, **i** ⇒ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:



- During address translation, if valid–invalid bit in page table entry is **I** ⇒ page fault

Page Table When Some Pages Are Not in Main Memory



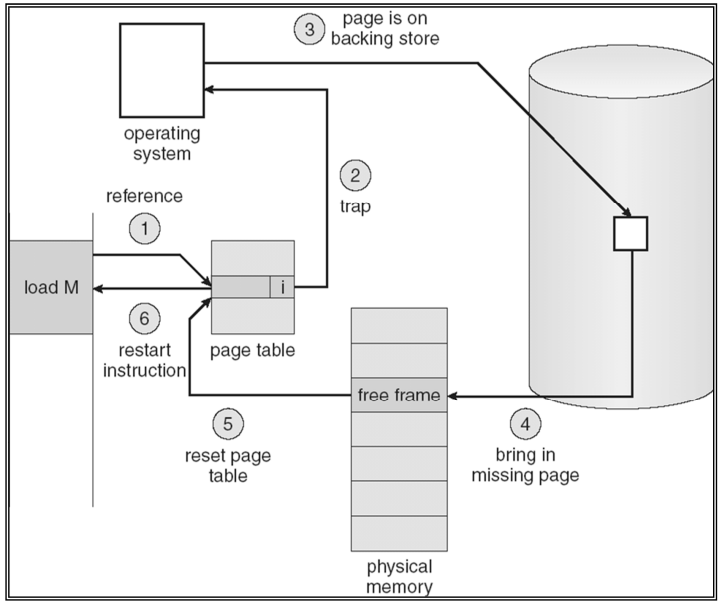
Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system:

  page fault

1. Operating system looks at another table to decide:
   - Invalid reference ⇒ abort
   - Just not in memory
2. Get empty frame
3. Swap page into frame
4. Reset tables
5. Set validation bit = **v**
6. Restart the instruction that caused the page fault

Steps in Handling a Page Fault



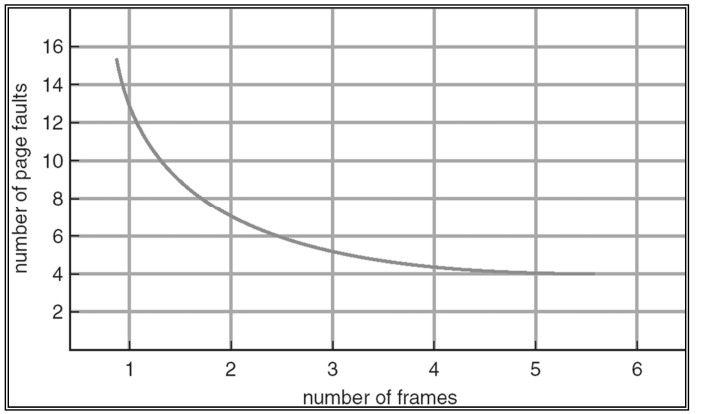
What happens if there is no free frame?

- Page replacement – find some page in memory, but not really in use, swap it out
  - algorithm
  - performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

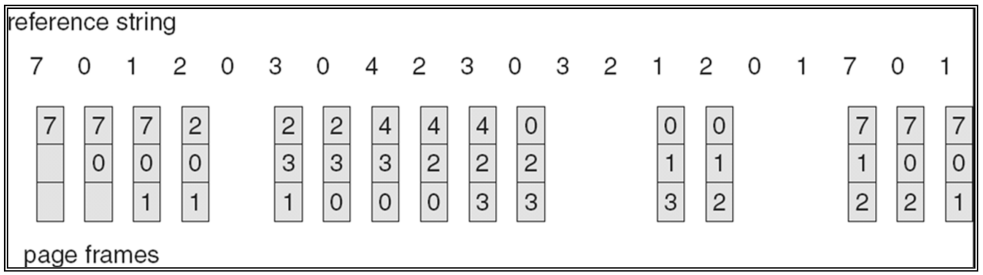
Page Replacement Algorithms

- Want lowest page-fault rate
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string

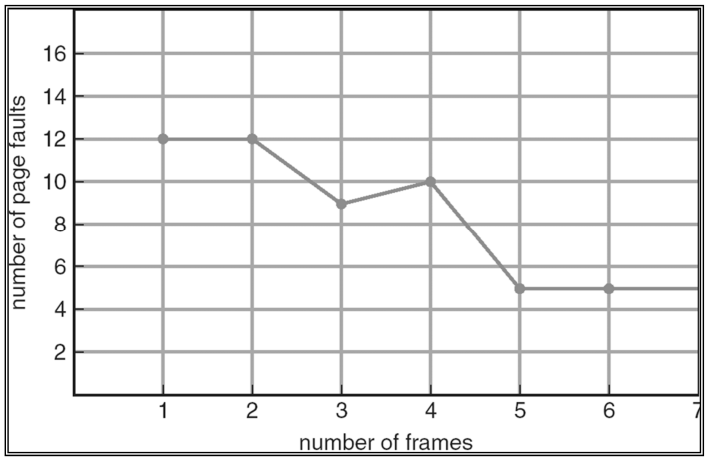
Graph of Page Faults Versus The Number of Frames



FIFO Page Replacement



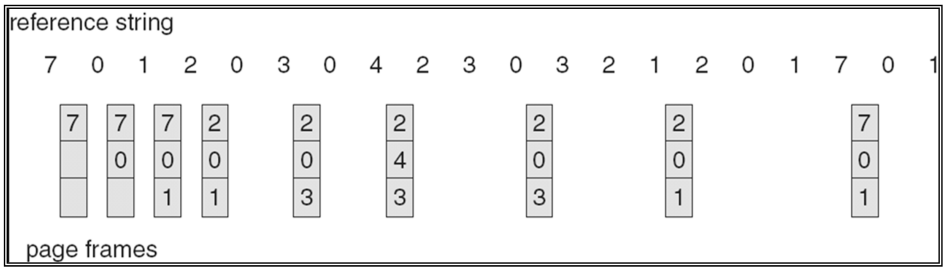
FIFO Illustrating Belady's Anomaly



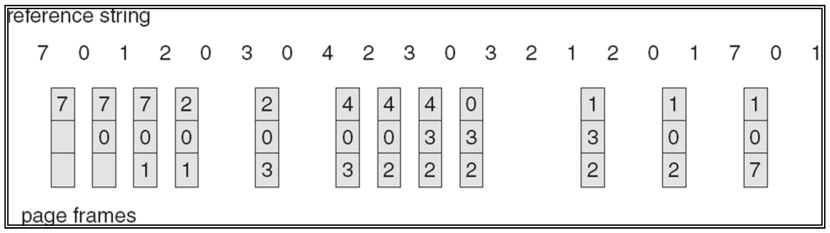
Optimal Algorithm

- Replace page that will not be used for longest period of time
- How do you know this?
- Used for measuring how well your algorithm performs

Optimal Page Replacement



LRU Page Replacement



Global vs. Local Allocation

- **Global replacement** – process selects a replacement frame from the set of all frames; one process can take a frame from another
- **Local replacement** – each process selects from only its own set of allocated frames

Thrashing

- **Thrashing** ≡ a process is busy swapping pages in and out
- If a process does not have “enough” pages, the page-fault rate is very high. This leads to:
  - low CPU utilization
  - operating system thinks that it needs to increase the degree of multiprogramming
  - another process added to the system

Thrashing (Cont.)

- Operation becomes inefficient
- Caused when a page is removed from memory but is called back shortly thereafter
- Can occur across jobs, when a large number of jobs are vying for a relatively few number of free pages
- Can happen within a job (e.g., in loops that cross page boundaries)

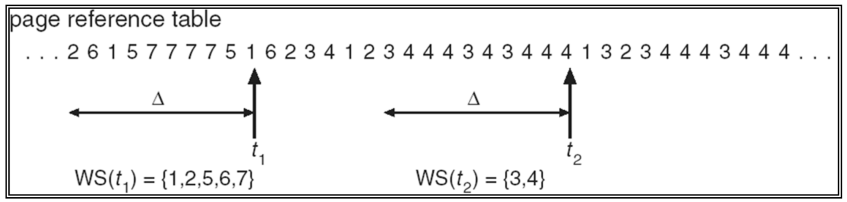
The Working Set

- **Working set:** Set of pages residing in memory that can be accessed directly without incurring a page fault
  - Improves performance of demand page schemes
  - Requires the concept of “locality of reference”
- **System must decide**
  - How many pages compose the working set
  - The maximum number of pages the operating system will allow for a working set

Working-Set Model

- $\Delta \equiv$ working-set window $\equiv$ a fixed number of page references
  Example: 10,000 instruction
- WSS_i (working set of Process P_i) =
  total number of pages referenced in the most recent Δ (varies in time)
  - if Δ too small will not encompass entire locality
  - if Δ too large will encompass several localities
  - if $\Delta = \infty \Rightarrow$ will encompass entire program
- $D = \Sigma\ WSS_i \equiv$ total demand frames
- if $D > m \Rightarrow$ Thrashing
- Policy if $D >$ m, then suspend one of the processes

Working-set model



Keeping Track of the Working Set

- Approximate with interval timer + a reference bit
- Example: Δ = 10,000
  - Timer interrupts after every 5000 time units
  - Keep in memory 2 bits for each page
  - Whenever a timer interrupts copy and sets the values of all reference bits to 0
  - If one of the bits in memory = 1 $\Rightarrow$ page in working set
- Why is this not completely accurate?
- Improvement = 10 bits and interrupt every 1000 time units

Program Structure

- Program structure
  - `int[128,128] data;`
  - Each row is stored in one page
  - Program 1

```
for (j = 0; j <128; j++)
     for (i = 0; i < 128; i++)
           data[i,j] = 0;
```

128 x 128 = 16,384 page faults

  - Program 2

```
for (i = 0; i < 128; i++)
     for (j = 0; j < 128; j++)
           data[i,j] = 0;
```

128 page faults